

BÀI THỰC HÀNH SỐ 2

I. Mục tiêu

Sau khi hoàn thành bài Lab, sinh viên có thể:

Kiến thức

Hiểu được vai trò và ý nghĩa của việc khai phá tập phổ biến và luật kết hợp trong khai phá dữ liệu. Nắm được cách phát hiện các mối quan hệ giữa các mục dữ liệu xuất hiện cùng nhau trong các giao dịch. Cụ thể, sinh viên hiểu được:

1. Khái niệm itemset, frequent itemset và luật kết hợp (association rule).
2. Ý nghĩa của các chỉ số đánh giá trong khai phá luật kết hợp như: Support, Confidence
3. Nguyên lý hoạt động của hai thuật toán khai phá tập phổ biến là: Apriori và FP-Growth
4. Cách sinh luật kết hợp từ các tập phổ biến và đánh giá chất lượng của các luật thu được.

Kỹ năng

- Xác định và biểu diễn dữ liệu giao dịch dưới dạng phù hợp cho bài toán khai phá tập phổ biến.
- Áp dụng các thuật toán Apriori và FP-Growth để tìm các tập phổ biến từ dữ liệu giao dịch. Tính toán các chỉ số support và confidence để đánh giá các luật kết hợp.
- Sinh và lựa chọn các luật kết hợp thỏa điều kiện min_sup và min_conf.
- Phân tích và diễn giải ý nghĩa của các luật kết hợp thu được trong ngữ cảnh dữ liệu.

II. Tóm tắt lý thuyết

1. Khái niệm tập phổ biến và luật kết hợp

Trong khai phá dữ liệu, tập phổ biến (Frequent Itemset) là tập hợp các item xuất hiện cùng nhau trong cơ sở dữ liệu giao dịch với tần suất lớn hơn hoặc bằng một ngưỡng tối thiểu gọi là **minimum support**.

Support của một tập mục là tỷ lệ số giao dịch chứa tập mục đó so với tổng số giao dịch.

$$support(X) = \frac{\text{số giao dịch chứa } X}{\text{tổng số giao dịch}}$$

Một luật kết hợp (Association Rule) có dạng:

$$X \rightarrow Y$$

Trong đó:

- X và Y là hai tập mục
- $X \cap Y = \emptyset$

Luật kết hợp thể hiện rằng: nếu một giao dịch chứa X thì có khả năng cao cũng chứa Y .

Độ tin cậy của luật được đo bằng confidence:

$$confidence(X \rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

Ví dụ:

Giả sử có các giao dịch:

TID	Items
T1	Bread, Milk
T2	Bread, Diaper, Beer
T3	Milk, Diaper
T4	Bread, Milk, Diaper

Nếu **min_sup = 50%**, tập mục {Bread, Milk} xuất hiện trong 2/4 giao dịch $\rightarrow$ support = 50% $\rightarrow$ là **tập phổ biến**.

Từ đó có thể tạo luật: Bread $\rightarrow$ Milk

Nếu 2 trong 3 giao dịch chứa Bread cũng chứa Milk thì: confidence = 2/3 = 66.7%

2. Giải thuật Apriori

Apriori là thuật toán dùng để tìm các tập mục phổ biến (frequent itemsets) trong dữ liệu giao dịch. Thuật toán dựa trên ý tưởng:

Nếu một tập mục là phổ biến thì mọi tập con của nó cũng phải là phổ biến.

Apriori hoạt động theo từng bước:

1. Đếm số lần xuất hiện của từng item để tìm 1-itemset phổ biến.
2. Từ các tập phổ biến này, tạo ra các candidate itemset lớn hơn (2-itemset, 3-itemset,...).
3. Tính support của các candidate.
4. Loại bỏ các tập không đạt min_sup.
5. Lặp lại quá trình cho đến khi không còn tập phổ biến mới.

Nhược điểm của Apriori là phải quét dữ liệu nhiều lần và sinh ra rất nhiều candidate.

3. Giải thuật FP-growth

FP-Growth là thuật toán tìm tập phổ biến hiệu quả hơn Apriori vì không cần sinh candidate itemsets.

Thuật toán hoạt động theo hai bước chính:

Bước 1: Xây dựng FP-Tree

- Quét dữ liệu để tìm các item phổ biến.
- Sắp xếp các item theo tần suất giảm dần.
- Xây dựng FP-Tree để nén dữ liệu giao dịch.

Bước 2: Khai thác FP-Tree

- Từ FP-Tree, xây dựng các conditional pattern base.
- Tạo các conditional FP-Tree.
- Từ đó tìm ra các tập phổ biến.

Nhờ cấu trúc FP-Tree, FP-Growth giảm số lần quét dữ liệu và thường nhanh hơn Apriori với dữ liệu lớn.

4. Tìm luật kết hợp dựa trên các tập phổ biến

Sau khi tìm được các **tập phổ biến**, ta có thể sinh ra **luật kết hợp** để phát hiện mối quan hệ giữa các item.

5. Ví dụ minh họa:

Cho bảng dữ liệu với 6 giao dịch như sau:

TID	Items
T001	A, B, D, E
T002	B, C, E
T003	A, B, D, E
T004	A, B, C, E
T005	A, B, C, D, E
T006	B, C, D

Với $\text{min_sup} = 50\%$ và $\text{min_conf} = 70\%$:

1. Tìm tất cả các tập phổ biến từ mẫu dữ liệu trên bằng giải thuật Apriori.
2. Tìm tất cả các tập phổ biến từ mẫu dữ liệu trên bằng giải thuật FP-growth.
3. Tìm tất cả các luật kết hợp dựa trên các tập phổ biến tìm được ở câu 1.

Câu 1:

Cơ sở dữ liệu gồm 6 giao dịch:

$T001 = \{A, B, D, E\}$, $T002 = \{B, C, E\}$, $T003 = \{A, B, D, E\}$, $T004 = \{A, B, C, E\}$,
 $T005 = \{A, B, C, D, E\}$, $T006 = \{B, C, D\}$.

Với $\text{min_sup} = 50\%$, điều này có nghĩa là một tập mục phải xuất hiện ít nhất 3 lần trong cơ sở dữ liệu thì mới được xem là tập phổ biến.

Bước 1: tìm các **1-itemset phổ biến**. Ta đếm số lần xuất hiện của từng item trong toàn bộ các giao dịch. Kết quả cho thấy:

- A: xuất hiện 4 lần
- B: xuất hiện 6 lần
- C: xuất hiện 4 lần
- D: xuất hiện 4 lần
- E: xuất hiện 5 lần

Vì tất cả các giá trị này đều lớn hơn hoặc bằng 3 nên tất cả các item đều là tập phổ biến 1 phần tử. Do đó ta có tập L1 gồm: {A}, {B}, {C}, {D}, {E}.

Bước 2: từ tập L1 ta sinh ra các **candidate 2-itemset** bằng cách kết hợp các item lại với nhau. Các tập được tạo ra gồm: {AB}, {AC}, {AD}, {AE}, {BC}, {BD}, {BE}, {CD}, {CE}, {DE}. Sau khi quét cơ sở dữ liệu để tính support cho từng tập, ta nhận được kết quả:

AB: xuất hiện 4 lần	BD: xuất hiện 3 lần
AC: xuất hiện 2 lần	BE: xuất hiện 5 lần
AD: xuất hiện 3 lần	CD: xuất hiện 2 lần
AE: xuất hiện 4 lần	CE: xuất hiện 3 lần
BC: xuất hiện 4 lần	DE: xuất hiện 3 lần

Theo điều kiện $\min_sup = 3$, các tập AC và CD bị loại vì support nhỏ hơn 3. Như vậy tập phổ biến 2 phần tử là: {A, B}, {A, D}, {A, E}, {B, C}, {B, D}, {B, E}, {C, E}, {D, E}. Tập này được ký hiệu là L2.

Bước 3: từ tập phổ biến 2 phần tử L2, ta tiến hành sinh các candidate 3-itemset bằng cách kết hợp các item trong L2. Từ các item {A, B, C, D, E}, các tổ hợp 3 phần tử có thể tạo ra gồm: {A, B, C}, {A, B, D}, {A, B, E}, {A, C, D}, {A, C, E}, {A, D, E}, {B, C, D}, {B, C, E}, {B, D, E}, {C, D, E}. Kết quả cho thấy:

ABC: xuất hiện 2 lần (T004, T005)
ABD: xuất hiện 3 lần (T001, T003, T005)
ABE: xuất hiện 4 lần (T001, T003, T004, T005)
ACD: xuất hiện 1 lần (T005)
ACE: xuất hiện 2 lần (T004, T005)
ADE: xuất hiện 3 lần (T001, T003, T005)
BCD: xuất hiện 2 lần (T005, T006)
BCE: xuất hiện 3 lần (T002, T004, T005)
BDE: xuất hiện 3 lần (T001, T003, T005)
CDE: xuất hiện 1 lần (T005)

Vì tất cả đều có support ≥ 3 nên chúng đều là các tập phổ biến 3 phần tử. Do đó ta có $L3 = \{A, B, D\}, \{A, B, E\}, \{A, D, E\}, \{B, C, E\}, \{B, D, E\}$.

Bước 4: từ các tập trong $L3$ ta tạo candidate 4-itemset. Tập hợp duy nhất thỏa điều kiện Apriori là $\{A, B, D, E\}$. Khi kiểm tra trong cơ sở dữ liệu, tập này xuất hiện 3 lần nên vẫn thỏa điều kiện min_sup.

Vì vậy ta có $L4 = \{ABDE\}$.

Sau bước này, không thể tạo thêm candidate 5-itemset nên thuật toán dừng lại.

Câu 2:

Bước 1: Đếm tần suất xuất hiện của các item

Trước tiên ta quét toàn bộ cơ sở dữ liệu để đếm số lần xuất hiện của từng item. Kết quả cho thấy A xuất hiện 4 lần, B xuất hiện 6 lần, C xuất hiện 4 lần, D xuất hiện 4 lần và E xuất hiện 5 lần. Vì tất cả đều lớn hơn hoặc bằng 3 nên tất cả các item đều là **frequent items**.

Sau đó các item được **sắp xếp theo tần suất giảm dần** để thuận tiện cho việc xây dựng FP-tree. Thứ tự là:

Item	Support count
B	6
E	5
A	4
C	4
D	4

Bước 2: Sắp xếp lại các giao dịch

Mỗi giao dịch được sắp xếp lại theo thứ tự tần suất vừa tìm được:

T001	→	B,	E,	A,	D	
T002	→	B,	E,	C		
T003	→	B,	E,	A,	D	
T004	→	B,	E,	A,	C	
T005	→	B,	E,	A,	C,	D
T006	→	B, C, D				

Việc sắp xếp này giúp các giao dịch có prefix giống nhau được chia sẻ trong FP-tree, từ đó làm giảm kích thước cây.

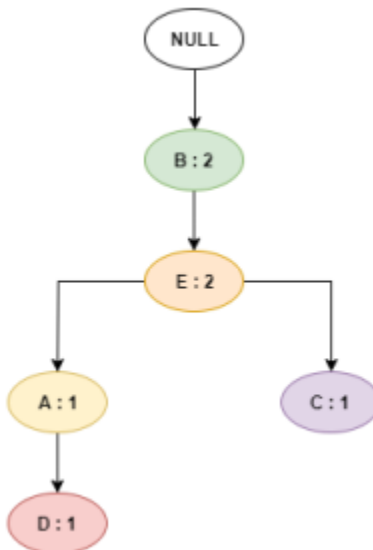
Bước 3: Xây dựng FP-tree

Ta bắt đầu từ **nút gốc (root)** và lần lượt chèn từng giao dịch vào cây.

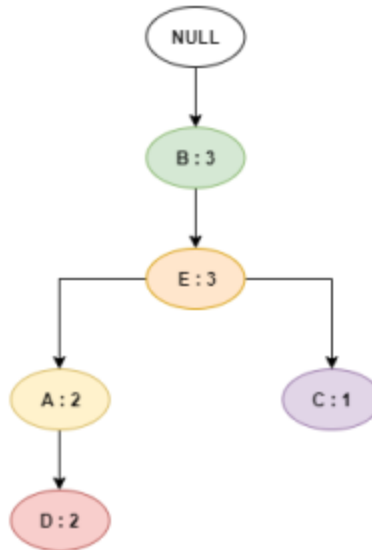
Giao dịch đầu tiên (B, E, A, D) tạo ra một nhánh từ root:



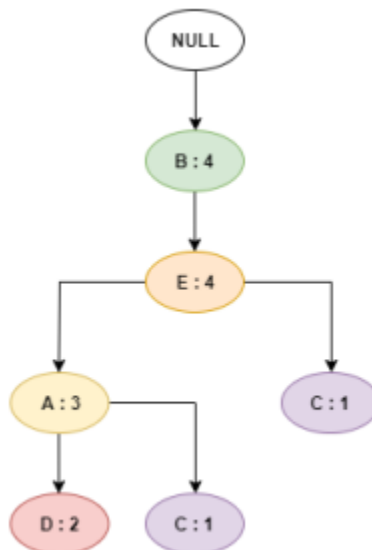
Giao dịch thứ hai (B, E, C) chia sẻ prefix B và E với nhánh trước, vì vậy ta tăng bộ đếm của B và E, sau đó tạo thêm nhánh C dưới E.



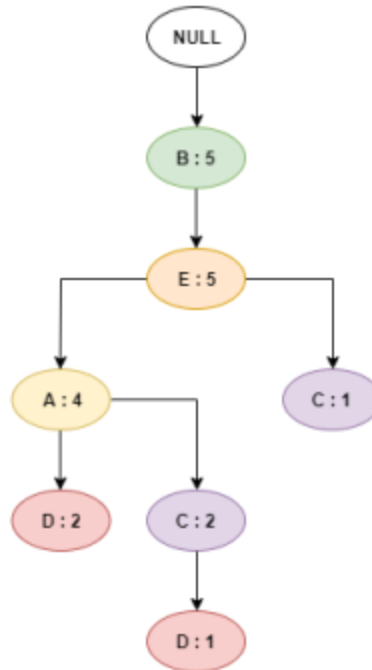
Giao dịch thứ ba (B, E, A, D) trùng với nhánh đã tồn tại nên chỉ cần tăng bộ đếm của các nút B, E, A và D.



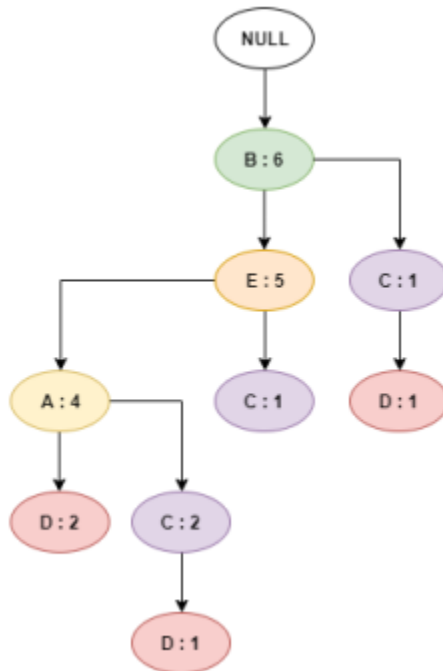
Giao dịch thứ tư (B, E, A, C) tiếp tục chia sẻ prefix B, E và A, sau đó tạo thêm nhánh C dưới A.



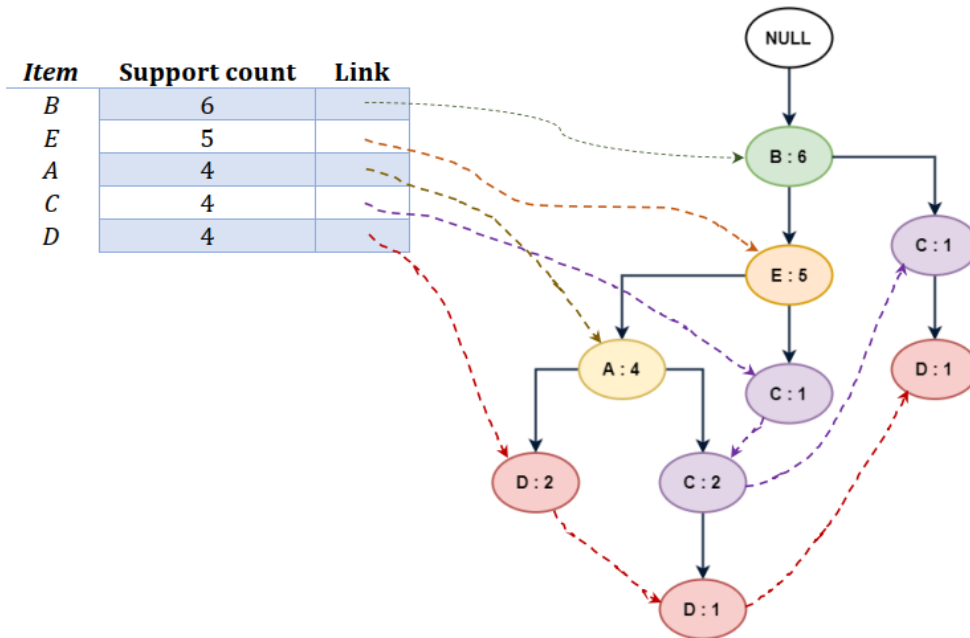
Giao dịch thứ năm (B, E, A, C, D) tiếp tục đi theo nhánh $B \rightarrow E \rightarrow A \rightarrow C$ và thêm nút D dưới C.



Giao dịch cuối cùng (B, C, D) chia sẻ prefix B với cây, sau đó tạo nhánh mới $C \rightarrow D$.



Sau khi chèn tất cả các giao dịch, ta thu được **FP-tree hoàn chỉnh** biểu diễn toàn bộ cơ sở dữ liệu giao dịch.



Bước 4: Khai thác các tập phổ biến từ FP-tree

Sau khi xây dựng xong FP-tree, giải thuật FP-Growth không sinh candidate như Apriori mà khai thác trực tiếp trên cây. Ý tưởng chính là: xét từng item làm hậu tố (suffix), sau đó tìm tất cả các mẫu phổ biến có chứa item đó.

Thuật toán thường xét các item theo thứ tự từ ít phổ biến hơn đến phổ biến hơn trong bảng header. Với dữ liệu này, thứ tự có thể xét là: D, C, A, E, B. Với mỗi item, ta thực hiện ba việc:

- (1) tìm conditional pattern base,
- (2) xây conditional FP-tree,
- (3) rút ra các tập phổ biến có chứa item đang xét.

Cơ sở mẫu điều kiện:

Item	Conditional pattern base	Support count
D	B, E, A	2
	B, E, A, C	1
	B, C	1
C	B, E	1
	B, E, A	2
	B	1
A	B, E	4
E	B	5
B	Null	

4.1. Khai thác với item D

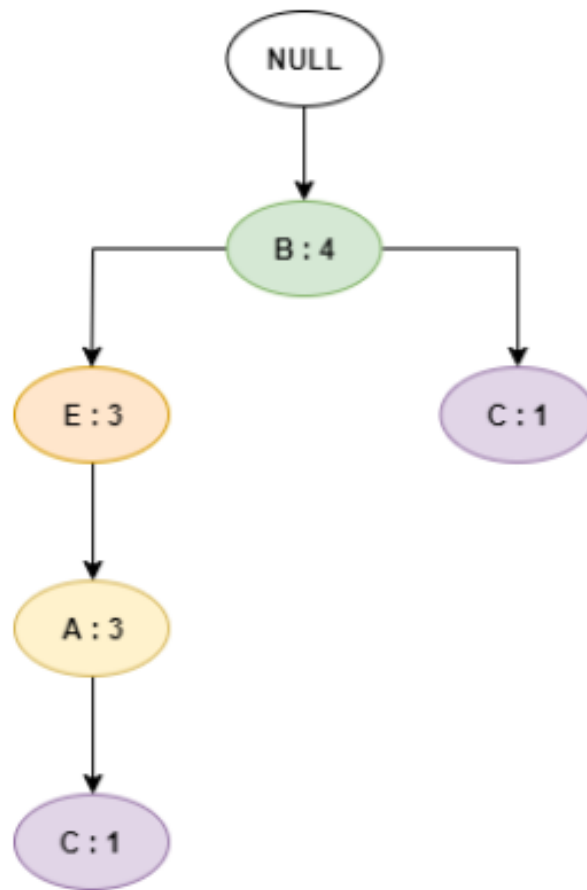
Trước hết, ta chọn D làm hậu tố. Ta tìm tất cả các đường đi trong FP-tree kết thúc tại nút D. Với mỗi nút D, ta lấy phần đường đi từ gốc đến ngay trước D; phần này được gọi là prefix path. Tập hợp các prefix path của D tạo thành conditional pattern base của D.

Từ FP-tree đã dựng, các đường đi dẫn đến D là:

- B, E, A, D với số đếm 2
- B, E, A, C, D với số đếm 1
- B, C, D với số đếm 1

Khi bỏ D ra khỏi cuối đường đi, ta thu được conditional pattern base của D là:

- B, E, A : 2
- B, E, A, C : 1
- B, C : 1



Bây giờ ta đếm lại tần suất của các item trong conditional pattern base này:

- B xuất hiện $2 + 1 + 1 = 4$
- E xuất hiện $2 + 1 = 3$
- A xuất hiện $2 + 1 = 3$
- C xuất hiện $1 + 1 = 2$

Vì **min_sup** = 3, item C không đạt ngưỡng nên bị loại. Chỉ còn lại B, E, A.

Từ đó, các tập phổ biến có chứa D là:

- {D} vì D xuất hiện 4 lần
- {B, D} vì B đi cùng D được 4 lần
- {E, D} vì E đi cùng D được 3 lần
- {A, D} vì A đi cùng D được 3 lần
- {B, E, D} vì B và E cùng đi với D được 3 lần
- {B, A, D} vì B và A cùng đi với D được 3 lần
- {A, E, D} vì A và E cùng đi với D được 3 lần
- {A, B, E, D} vì A, B, E cùng đi với D được 3 lần

Như vậy, khi khai thác theo hậu tố D, ta thu được toàn bộ các mẫu phổ biến có chứa D.

4.2. Khai thác với item C

Tiếp theo, chọn C làm hậu tố. Ta tìm tất cả các đường đi kết thúc tại C trong FP-tree. Các đường đi đó là:

- B, E, C với số đếm 1
- B, E, A, C với số đếm 2
- B, C với số đếm 1

Bỏ C ra, ta có conditional pattern base của C:

- B, E : 1
- B, E, A : 2
- B : 1



Đếm lại tần suất trong conditional pattern base của C:

- B xuất hiện $1 + 2 + 1 = 4$
- E xuất hiện $1 + 2 = 3$
- A xuất hiện 2

Do $\text{min_sup} = 3$ nên A bị loại, còn B và E được giữ lại.

Từ đó, các tập phổ biến có chứa C là:

- {C}
- {B, C}
- {E, C}
- {B, E, C}

Trong đó:

- $\text{support}(C) = 4$
- $\text{support}(BC) = 4$
- $\text{support}(CE) = 3$
- $\text{support}(BCE) = 3$

4.3. Khai thác với item A

Tiếp theo, xét A làm hậu tố. Các nút A trong FP-tree chỉ nằm trên cùng một nhánh chính:

- B, E, A với số đếm 4

Bỏ A ra, ta có conditional pattern base của A:

- B, E : 4



Đếm lại:

- B xuất hiện 4
- E xuất hiện 4

Cả hai đều đạt min_sup. Vì vậy các tập phổ biến có chứa A là:

- {A}
- {A, B}
- {A, E}
- {A, B, E}

Tương ứng:

- $\text{support}(A) = 4$
- $\text{support}(AB) = 4$
- $\text{support}(AE) = 4$
- $\text{support}(ABE) = 4$

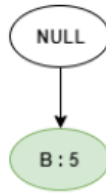
4.4. Khai thác với item E

Xét E làm hậu tố. Từ FP-tree, E xuất hiện trên đường:

- B, E với số đếm 5

Bỏ E ra, conditional pattern base của E là:

- B : 5



Do đó các tập phổ biến có chứa E là:

- {E}
- {B, E}

với:

- $\text{support}(E) = 5$
- $\text{support}(BE) = 5$

4.5. Khai thác với item B

Cuối cùng xét **B**. Vì B là item đứng đầu các nhánh, khi bỏ B ra thì không còn prefix path nào phía trước. Do đó B chỉ tạo ra tập phổ biến đơn:

- {B}

với $\text{support}(B) = 6$.

Câu 3:

Các tập phổ biến gồm:

Các 1-itemset: {A}, {B}, {C}, {D}, {E};

Các 2-itemset: {A, B}, {A, D}, {A, E}, {B, C}, {B, D}, {B, E}, {C, E}, {D, E};

Các 3-itemset: {A, B, D}, {A, B, E}, {A, D, E}, {B, C, E}, {B, D, E};

4-itemset: {A, B, D, E}.

Một luật kết hợp có dạng:

$$X \rightarrow Y$$

với độ tin cậy:

$$\text{conf}(X \rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$$

Ta chỉ giữ lại những luật có confidence $\geq 70\%$.

Sinh luật từ các tập phổ biến 2 phần tử

Từ tập $\{A, B\}$, ta có:

- $A \rightarrow B = 4/4 = 100\%$, giữ lại.
- $B \rightarrow A = 4/6 = 66.7\%$, loại.

Từ tập $\{A, D\}$, , ta có:

- $A \rightarrow D = 3/4 = 75\%$, giữ lại.
- $D \rightarrow A = 3/4 = 75\%$, giữ lại.

Từ tập $\{A, E\}$, , ta có:

- $A \rightarrow E = 4/4 = 100\%$, giữ lại.
- $E \rightarrow A = 4/5 = 80\%$, giữ lại.

Từ tập $\{B, C\}$, , ta có:

- $B \rightarrow C = 4/6 = 66.7\%$, loại.
- $C \rightarrow B = 4/4 = 100\%$, giữ lại.

Từ tập $\{B, D\}$, , ta có:

- $B \rightarrow D = 4/6 = 66.7\%$, loại.
- $D \rightarrow B = 4/4 = 100\%$, giữ lại.

Từ tập $\{B, E\}$, , ta có:

- $B \rightarrow E = 5/6 = 83.3\%$, giữ lại.
- $E \rightarrow B = 5/5 = 100\%$, giữ lại.

Từ tập $\{C, E\}$, , ta có:

- $C \rightarrow E = 3/4 = 75\%$, giữ lại.
- $E \rightarrow C = 3/5 = 60\%$, loại.

Từ tập $\{D, E\}$, ta có:

- $D \rightarrow E = 3/4 = 75\%$, giữ lại.
- $E \rightarrow D = 3/5 = 60\%$, loại.

Sinh luật từ các tập phổ biến 3 phần tử

Từ tập $\{A, B, D\}$,

Ta có:

- $AB \rightarrow D = 3/4 = 75\%$, giữ lại.
- $AD \rightarrow B = 3/3 = 100\%$, giữ lại.
- $BD \rightarrow A = 3/4 = 75\%$, giữ lại.
- $A \rightarrow BD = 3/4 = 75\%$, giữ lại.
- $D \rightarrow AB = 3/4 = 75\%$, giữ lại.
- $B \rightarrow AD = 3/6 = 50\%$, loại.

Từ tập $\{A, B, E\}$,

Ta có:

- $AB \rightarrow E = 4/4 = 100\%$, giữ lại.
- $AE \rightarrow B = 4/4 = 100\%$, giữ lại.
- $BE \rightarrow A = 4/5 = 80\%$, giữ lại.
- $A \rightarrow BE = 4/4 = 100\%$, giữ lại.
- $E \rightarrow AB = 4/5 = 80\%$, giữ lại.

Từ tập $\{A, D, E\}$,

Ta có:

- $AD \rightarrow E = 3/3 = 100\%$, giữ lại.
- $AE \rightarrow D = 3/4 = 75\%$, giữ lại.
- $DE \rightarrow A = 3/3 = 100\%$, giữ lại.
- $A \rightarrow DE = 3/4 = 75\%$, giữ lại.

- $D \rightarrow AE = 3/4 = 75\%$, giữ lại.

Từ tập $\{B, C, E\}$,

Ta có:

- $BC \rightarrow E = 3/4 = 75\%$, giữ lại.
- $CE \rightarrow B = 3/3 = 100\%$, giữ lại.
- $C \rightarrow BE = 3/4 = 75\%$, giữ lại.

Từ tập $\{B, D, E\}$,

Ta có:

- $BD \rightarrow E = 3/4 = 75\%$, giữ lại.
- $DE \rightarrow B = 3/3 = 100\%$, giữ lại.
- $D \rightarrow BE = 3/4 = 75\%$, giữ lại.

Sinh luật từ tập phổ biến 4 phần tử $\{A, B, D, E\}$

Từ tập này, ta xét tất cả các tập con khác rỗng và khác toàn bộ tập.

Các luật đạt điều kiện confidence là:

- $A \rightarrow BDE = 3/4 = 75\%$
- $D \rightarrow ABE = 3/4 = 75\%$
- $AB \rightarrow DE = 3/4 = 75\%$
- $AD \rightarrow BE = 3/3 = 100\%$
- $AE \rightarrow BD = 3/4 = 75\%$
- $BD \rightarrow AE = 3/4 = 75\%$
- $DE \rightarrow AB = 3/3 = 100\%$
- $ABD \rightarrow E = 3/3 = 100\%$
- $ABE \rightarrow D = 3/4 = 75\%$
- $ADE \rightarrow B = 3/3 = 100\%$
- $BDE \rightarrow A = 3/3 = 100\%$

IV. Thực hành

Bài tập 1

Cho file *groceries.csv* là thông tin về các giao dịch trong một tháng tại một cửa hàng. Hãy sử dụng Python thực hiện các yêu cầu dưới đây:

Câu 1: Đọc *groceries.csv* dataset và chuyển thành danh sách các giao dịch. In ra số lượng giao dịch và số lượng item.

Gợi ý:

```
with open("groceries.csv", "r") as f:
```

```
    for line in f:
```

```
        items = line.strip().split(",")
```

```
        transactions.append(items)
```

Câu 2: Tính số giao dịch, số item khác nhau và top 10 item xuất hiện nhiều nhất.

Gợi ý:

Số item khác nhau:

```
num_transactions = len(transactions)
```

```
all_items = [item for t in transactions for item in t]
```

```
unique_items = set(all_items)
```

```
counter = Counter(all_items)
```

Top 10 item: `counter.most_common(10)`

Câu 3: Viết code cài đặt tính support của từng 1-itemset.

Gợi ý:

```
for item in unique_items:
```

```
    count = sum(1 for t in transactions if item in t)
```

```
    support_1[item] = count / num_transactions
```

Câu 4: Tìm frequent 1-itemsets với một ngưỡng min_support cho trước.

Gợi ý:

$L1 = \{item:sup \text{ for } item, sup \text{ in } support_1.items() \text{ if } sup \geq min_support\}$

Câu 5: Viết code cài đặt để sinh candidate 2-itemsets và tìm frequent 2-itemsets.

Gợi ý:

```
items_L1 = list(L1.keys())
C2 = list(combinations(items_L1,2))
support_2 = {}
for pair in C2:
    count = sum(1 for t in transactions if set(pair).issubset(t))
    support_2[pair] = count/num_transactions
L2 = {pair:sup for pair,sup in support_2.items() if sup >= min_support}
```

Câu 6: Không sử dụng thư viện mlxtend, cài đặt Apriori bằng Python. Viết hàm có dạng: **apriori(transactions, min_support)**

Gợi ý: Tham khảo lại thuật toán.

Câu 7: In ra các bước trung gian C1, L1, C2, L2, ... của Apriori.

Gợi ý:

```
L = apriori(transactions,0.01)
for i,l in enumerate(L):
    print(f"L{i+1}")
    for item,sup in l.items():
        print(item,sup)
    print()
```

Câu 8: Dùng TransactionEncoder để one-hot encode dữ liệu.

Gợi ý:

```
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array,columns=te.columns_)
```

Câu 9: Dùng thư viện *mlxtend* chạy Apriori.

Gợi ý code:

```
frequent_itemsets = apriori(df,  
                             min_support=0.01,  
                             use_colnames=True)
```

Câu 10: Dùng thư viện *mlxtend* chạy FP-Growth.

Gợi ý:

```
fp_itemsets = fpgrowth(df,  
                        min_support=0.01,  
                        use_colnames=True)
```

```
print(fp_itemsets.head())
```

Câu 11: Dùng thư viện để lấy ra *association rules* từ *frequent itemsets*.

Gợi ý:

```
rules = association_rules(fp_itemsets,  
                           metric="confidence",  
                           min_threshold=0.2)
```

Câu 12: Lọc top 10 luật theo confidence.

Gợi ý:

```
rules_sorted = rules.sort_values(by="confidence",ascending=False)
```

Câu 13: Tìm các luật có consequent là whole milk.

Gợi ý:

```
rules_milk = rules[rules['consequents'].apply(lambda x: 'whole milk' in x)]
```

Câu 14: Khảo sát ảnh hưởng của *min_support* và *min_confidence* lên số lượng itemsets/rules.

Gợi ý:

```
supports=[0.01,0.02,0.03,0.05]
```

```
counts=[]
```

for s in supports:

```
itemsets = apriori(df,min_support=s,use_colnames=True)
```

```
counts.append(len(itemsets))
```

Bài tập 2

Cho file *Online_Retail.csv* là thông tin về các giao dịch trong một tháng tại một cửa hàng. Hãy sử dụng Python thực hiện các yêu cầu dưới đây:

Câu 1: Đọc file *online_retail.xlsx* và hiển thị thông tin cơ bản của dataset. Làm sạch dữ liệu: Loại bỏ các dòng có CustomerID bị thiếu và chỉ giữ các dòng có: Quantity > 0, UnitPrice > 0

Câu 2: Chuyển dữ liệu về dạng **transaction list**, trong đó mỗi InvoiceNo là một transaction.

Gợi ý: Dùng: `groupby("InvoiceNo")`

Câu 3: Viết hàm tính **support** của một itemset.

$$Support(X) = \frac{\text{số transaction chứa X}}{\text{tổng transaction}}$$

Gợi ý: Duyệt từng transaction

Dùng: `set(itemset).issubset(set(transaction))`

Câu 4: Tính support của một số item cụ thể.

Gợi ý: Gọi lại hàm ở Câu 3. Thử với:

- 1-item
- 2-item

Câu 5: Tìm frequent 1-itemsets với min_support = 0.02.

Gợi ý:

Duyệt tất cả item

Tính support

Lọc theo ngưỡng

Câu 6: Sinh candidate 2-itemsets từ frequent 1-itemsets.

Gợi ý: Dùng combinations(items, 2) hoặc union của itemsets

Câu 7: Tính support và tìm frequent 2-itemsets.

Gợi ý:

Dùng lại hàm support

Lọc theo min_support

Câu 8: Viết hàm tính confidence của luật:

$$X \rightarrow Y$$

Với X là 85123A và Y là 71053.

Gợi ý: Công thức: sử dụng công thức $\text{support}(X \cup Y) / \text{support}(X)$

Câu 9: Không sử dụng thư viện, cài đặt thuật toán Apriori để tìm tất cả frequent itemsets.

Câu 10: Chuyển transaction data sang dạng one-hot encoding.

Gợi ý: Dùng *TransactionEncoder*. Output: DataFrame dạng 0/1

Câu 11: Sử dụng thư viện mlxtend để tìm các tập phổ biến bằng thuật toán Apriori.

Gợi ý: Import: *from mlxtend.frequent_patterns import apriori*

Input là DataFrame one-hot

Câu 12: Sử dụng thư viện mlxtend để tìm các tập phổ biến bằng thuật toán FP-Growth.

Gợi ý: Import: *fpgrowth*

Câu 13: Tạo các association rules từ frequent itemsets tìm được ở câu 11 (Apriori).

Gợi ý: Dùng: *association_rules()*

Câu 14: Lọc các luật mạnh với: $\text{confidence} \geq 0.7$ và $\text{lift} \geq 1.2$

Gợi ý: dùng Filter DataFrame với kết quả đã có ở trên

Câu 15: Vẽ biểu đồ thể hiện min_support từ 0.01 đến 0.1 (bước nhảy = 0.01) và số lượng itemsets tương ứng